Quadrotor RANFTSMC Control System

Robust Adaptive Nonsingular Fast Terminal Sliding Mode Control for UAV Trajectory Tracking

Ghaith Chamaa & Muhammad Ureed Hussain
Under Supervision of Amine Abadi & Imtiaz Ur Rehman
Université Bourgogne Europe

January 11, 2026

Overview

1. Introduction
2. Mathematical Model
3. Problem Statement
4. Methodology (Control Design)
5. Results
6. Discussion
7. References

Introduction: Context

Quadrotor UAVs are increasingly used for surveillance, delivery, and inspection. However, control is challenging due to:

- **Under-actuation**: 6 Degrees of Freedom (DOF) but only 4 control inputs.
- **Nonlinearity**: Strong coupling between attitude and position dynamics.
- **Uncertainties**: Aerodynamic effects, payload mass variations, and sensor noise.
- **External Disturbances**: Wind gusts (d_i) can destabilize the vehicle.

Objective: Implement a control strategy that ensures finite-time convergence and robustness against these uncertainties.

Introduction: The Solution (RANFTSMC)

This project implements the **Robust Adaptive Nonsingular Fast Terminal Sliding Mode Control** based on Labbadi & Cherkaoui (2020).

Key Advantages of RANFTSMC

- **Nonsingular**: Avoids singularity problems found in standard Terminal SMC.
- **Fast Terminal**: Guarantees convergence in finite time (not asymptotic).
- **Adaptive**: Estimates the upper bounds of disturbances online (no need for prior knowledge of wind speed).
- **Continuous**: Uses smooth functions (tanh) to reduce "chattering" (high-frequency vibration).

Mathematical Model: Translational Dynamics

Translational Subsystem (Position):

$$\begin{cases} \ddot{x} = \frac{U_T}{m}(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi) + \underbrace{-\frac{K_x}{m}\dot{x}}_{f_x(\mathbf{x})} + d_x \\ \ddot{y} = \frac{U_T}{m}(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi) + \underbrace{-\frac{K_y}{m}\dot{y}}_{f_y(\mathbf{x})} + d_y \\ \ddot{z} = \frac{U_T}{m}(\cos\phi\cos\theta) + \underbrace{-g - \frac{K_z}{m}\dot{z}}_{f_z(\mathbf{x})} + d_z \end{cases}$$

Note: K_x, K_y, K_z are the **Aerodynamic Friction Coefficients**.

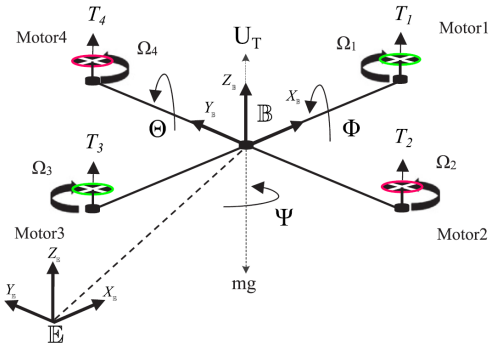

Figure 1: Quadrotor Configuration

Mathematical Model: Rotational Dynamics

Rotational Subsystem (Attitude):

$$\begin{cases} \ddot{\Phi} = \underbrace{\frac{1}{J_{xx}}(\dot{\Theta}\dot{\Psi}(J_{yy}-J_{zz}) - J_r\dot{\Theta}\varpi - K_\Phi\dot{\Phi}^2)}_{f_\Phi(\mathbf{x})} + \frac{1}{J_{xx}}U_\Phi + d_\Phi \\ \ddot{\Theta} = \underbrace{\frac{1}{J_{yy}}(\dot{\Phi}\dot{\Psi}(J_{zz}-J_{xx}) - J_r\dot{\Phi}\varpi - K_\Theta\dot{\Theta}^2)}_{f_\Theta(\mathbf{x})} + \frac{1}{J_{yy}}U_\Theta + d_\Theta \\ \ddot{\Psi} = \underbrace{\frac{1}{J_{zz}}(\dot{\Phi}\dot{\Theta}(J_{xx}-J_{yy}) - K_\Psi\dot{\Psi}^2)}_{f_\Psi(\mathbf{x})} + \frac{1}{J_{zz}}U_\Psi + d_\Psi \end{cases}$$

Note: $J_{xx/yy/zz}$: Moments of Inertia — J_r: Rotor Inertia — $K_{\Phi/\Theta/\Psi}$: Aerodynamic Friction

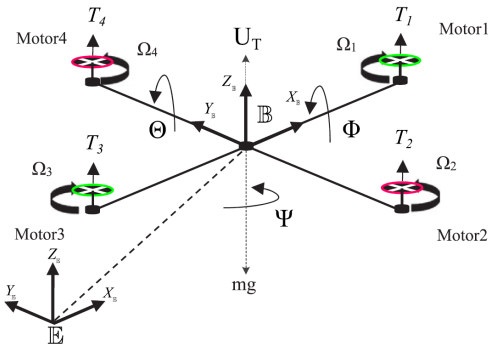

Figure 2: Quadrotor Configuration

Problem Statement

Control Objective

Design a control law that forces the system states $(x, y, z, \phi, \theta, \psi)$ to track a desired trajectory $(x_d, y_d, z_d, \phi_d, \theta_d, \psi_d)$ such that the tracking error $e(t) \to 0$ in finite time T_{final}.

Constraints & Challenges:

1. **Unknown Disturbances**: The disturbance $d(t)$ is bounded by $|d(t)| \leq \delta$, but the upper bound δ is unknown.
2. **Under-actuation**: x and y motions are generated by tilting the drone (coupling via ϕ and θ).
3. **Hardware Constraints**: The DJI Tello has limited thrust and relies on elevation Z via barometer for state estimation (noisy).

Methodology: Control Architecture

The controller is split into a Cascade Structure (Inner/Outer Loop).

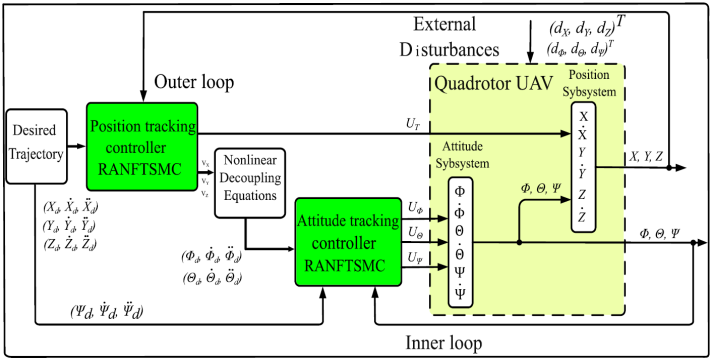

Figure 3: General Control Scheme (Labbadi & Cherkaoui, 2020)

- **Outer Loop**: Calculates U_T and virtual controls (V_X, V_Y and V_Z) for position.
- **Inner Loop**: Generates Torques (U_ϕ, U_θ, U_ψ) to track angles.

Methodology: The Sliding Surface

To ensure fast convergence without singularity, the Nonlinear Nonsingular Fast Terminal Sliding Surface is used:

$$\sigma = e + b_1|e|^{\alpha}\text{sgn}(e) + b_2|\dot{e}|^{\beta}\text{sgn}(\dot{e})$$

Where:

- $e = x - x_d$ is the tracking error.
- $1 < \beta < 2$ and $\alpha > \beta$.
- This structure ensures that when $\sigma = 0$, the error e converges to zero in finite time.

Methodology: Total Adaptive Control Law

The final control input U is the sum of the **Equivalent Control** (model-based) and the **Switching Control** (robustness).

Total Control Input

$$U(t) = U_{eq}(t) + U_{sw}(t)$$

- **1. Equivalent Control (U_{eq})**: Derived by setting $\dot{\sigma} = 0$ to ensure convergence along the sliding surface without disturbances.

$$U_{eq} = \underbrace{-f(\mathbf{x}) + \ddot{x}_d}_{\text{Dynamics Cancellation}} \underbrace{- \frac{1}{\beta b_2} |\dot{e}|^{2-\beta} \left(1 + \alpha b_1 |e|^{\alpha-1}\right) \operatorname{sgn}(\dot{e})}_{\text{Finite-Time Convergence Term}}$$

- **2. Adaptive Switching Control (U_{sw})**: Handles unknown disturbances (δ).

$$U_{sw} = -c\sigma - \underbrace{(\hat{a}_0 + \hat{a}_1 |e| + \hat{a}_2 |\dot{e}|)}_{\text{Adaptive Gain } \hat{K}} \tanh\left(\frac{\sigma}{\epsilon}\right)$$

Where $f(\mathbf{x})$ is the intrinsic physics (e.g., for Z-axis: $f(\mathbf{x}) = -g - \frac{K_z}{m}\dot{z}$).

Implementation Overview

The project consists of two implementations:

1. **Python Simulation**: Full physics simulation testing specific paper scenarios (Circle, Square, Complex).
2. **Hardware (DJI Tello)**: Axis-specific RANFTSMC implementation using ‘djitellopy‘ for communication and state estimation.

Simulation Setup: Disturbances & Trajectory

Disturbances

Time-Varying Disturbance: Simulating wind gusts (applied to acceleration):

$$\begin{cases} d_x = -0.8\sin(0.10t - 3.04) \\ \qquad +0.4\sin(0.44t - 13.46) \\ \qquad +0.08\sin(1.57t - 15\pi) \\ \qquad +0.05\sin(0.28t - 8.56)) \quad t \in [10, 30] \\ d_y = 0.5\sin(0.4t) + 0.5\cos(0.7t) \quad t \in [10, 50] \\ d_z = 0.5\cos(0.7t) \quad t \in [0, 80] \end{cases}$$

Constant Disturbance:
Step inputs of magnitude 1.0 applied sequentially to axes.

Trajectory (X_d, Y_d, Z_d, ψ_d)

Complex Trajectory:

$$X_d = \begin{cases} 0.5\cos(t/2) & t \in [0, 4\pi) \\ 0.5 & t \in [4\pi, 20) \\ 0.25t - 4.5 & t \in [20, 30) \\ 3 & t \in [30, 80] \end{cases}$$

$$Y_d = \begin{cases} 0.5\sin(t/2) & t \in [0, 4\pi) \\ 0.25t - 3.14 & t \in [4\pi, 20) \\ 5 - \pi & t \in [20, 30) \\ -0.23t + 8.9 & t \in [30, 40] \\ -0.5 & t \in [40, 80] \end{cases}$$

$$Z_d = \begin{cases} 0.125t + 1 & t \in [0, 4\pi) \\ 0.5\pi + 1 & t \in [4\pi, 40) \\ e^{-0.2t+8.9} & t \in [40, 80] \end{cases} \qquad \psi_d = 0rad$$

Simulation Result: Complex Trajectory

Scenario: "Complex" trajectory without disturbance.

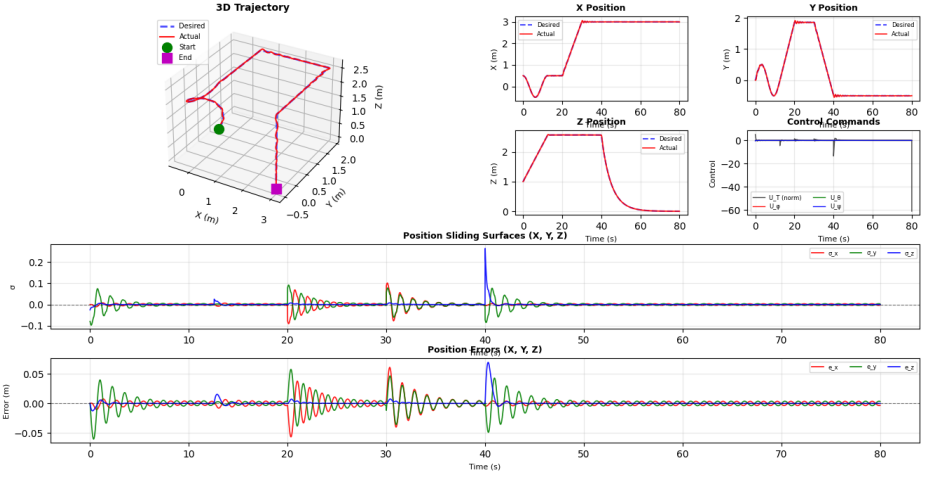

Figure 4: 3D Trajectory without Disturbance

Simulation Result: Complex Trajectory (Cont'd)

Scenario: "Complex" trajectory with constant disturbance.

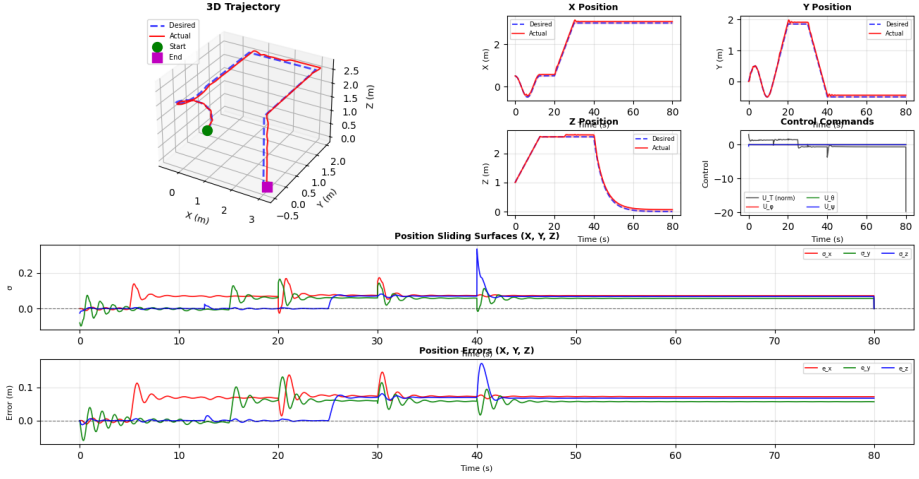

Figure 5: 3D Trajectory with constant disturbance

Simulation Result: Complex Trajectory (Cont'd)

Scenario: "Complex" trajectory with time varying disturbance.

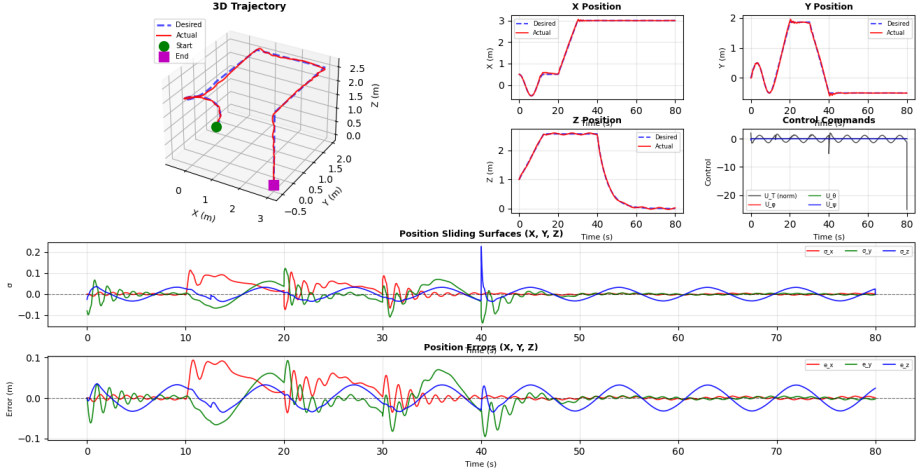

Figure 6: 3D Trajectory with time varying disturbance

Hardware Implementation: Why not ROS?

Why we did not use ROS 2 or Gazebo:

- **Hardware Availability**: There are currently very few off-the-shelf drones available in the lab that support ROS 2 natively.
- **Driver Instability**: We evaluated the existing ROS 2 package for the DJI Tello, but it was found to be unstable.
- **Strategic Decision**: Consequently, we utilized the native **Python library** (`djitellopy`). This ensured a stable, direct connection to the hardware without middleware overhead.

Hardware Implementation: DJI Tello

Challenges:

- High latency (WiFi communication - danger in losing communication).
- Noisy state estimation (Elevation Barometer sensitive to ground reflectance).
- Blind Positioning (no actual position readings).
- Different physical model params from the drone used in the paper (extreme necessity for fine tuning).
- Safety constraints (Limited workspace).

Solution: "Hybrid" Axis-Specific Controller.

- Independent RANFTSMC tuning for X, Y, and Z axes.
- Dead Reckoning (use of IMU velocities to get a pose estimation, assuming initial pose (0, 0, 0)).
- Direct Velocity Command generation.
- Workspace Bounding (Virtual Cage).

Figure 7: The IR sensor on the bottom of the drone

Hardware Implementation: DJI Tello IMU Calibration

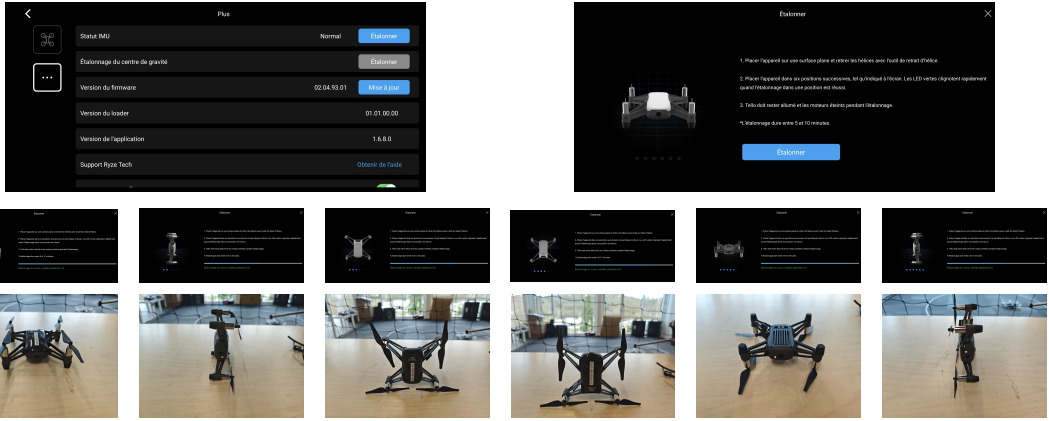

Figure 8: IMU Calibration Procedure (6-sided calibration)

Hardware Results

Flight Data (Workspace Size = 0.3m, Duration = 45s):

- **Control Rate**: ≈ 80 Hz.
- **ISE Metric**:
  - X: 0.0432
  - Y: 0.0562
  - Z: 0.1533

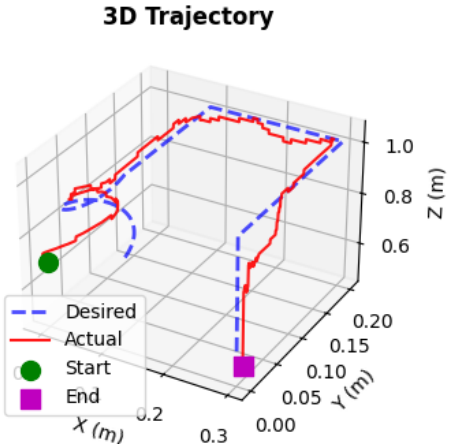

Figure 9: Hardware Flight Result (Desired vs Actual)

Hardware Results (Cont'd)

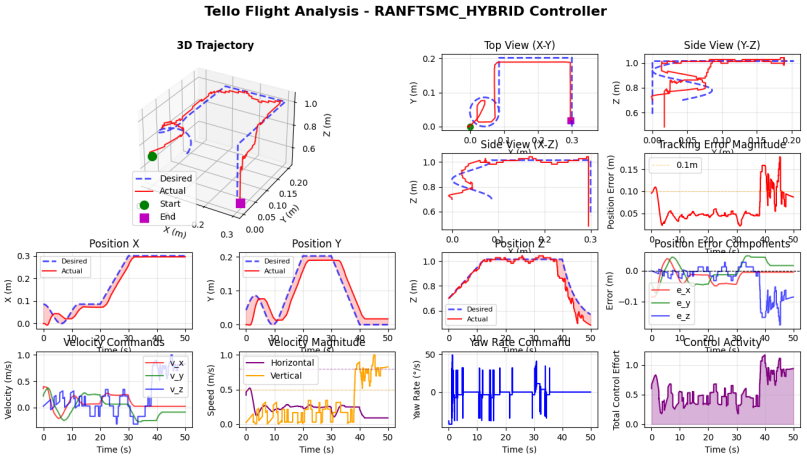

Figure 10: Hardware Experiment: Trajectory Tracking

Discussion

Strengths

- **Robustness**: Simulation showed excellent rejection of "Aggressive" disturbances.
- **Finite Time**: Error converges to zero faster than asymptotic methods (PID/LQR).
- **No Chattering**: The smooth tanh function prevented high-frequency vibration in motors.

Limitations

- **Tuning Complexity**: RANFTSMC has many parameters ($\alpha, \beta, \mu_{0,1,2}, b, c$) per axis.
- **Hardware Noise**: Differentiating noisy position data to get acceleration for the sliding surface is difficult on low-cost hardware.

References

M. Labbadi and M. Cherkaoui (2020).
Robust adaptive nonsingular fast terminal sliding-mode tracking control for an uncertain quadrotor UAV subjected to disturbances.
ISA Transactions, 99, 290–304.

Thank You